

Percolation : étude d'une transition de phase

Balthazar RICHARD

N° SCEI : 35034

« Transition, transformation, conversion. »

- Comment caractériser un phénomène de seuil ?
- Comment modéliser des phénomènes de seuil ?
- Quelle est la valeur de la probabilité critique d'un réseau élémentaire ?

Un exemple de percolation

Histoire et définitions

Phénomène de seuil

Ouverture

Du grain de café au grain de sable

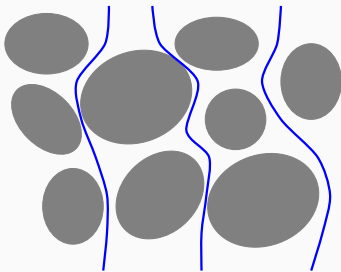


Café moulu dans un percolateur

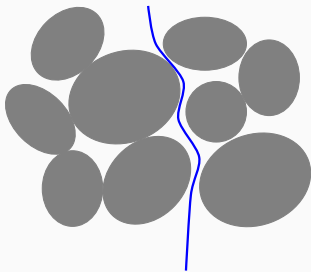


Roche poreuse

Du grain de café au grain de sable

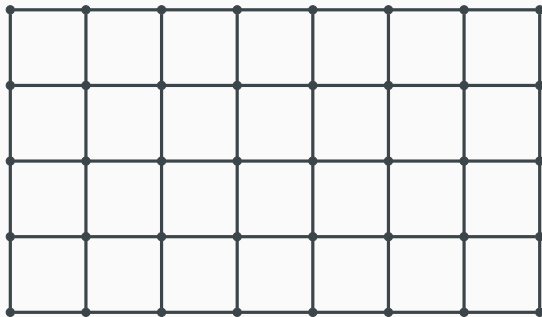


Milieu perméable

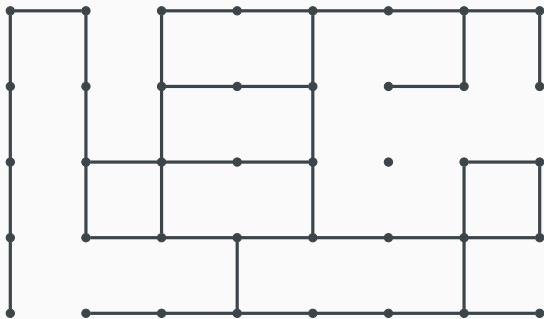


Milieu plus compact

Du grain de café au grain de sable



Du grain de café au grain de sable



Du grain de café au grain de sable

$$p = 0,229$$



Du grain de café au grain de sable

4% de la roche parcourue



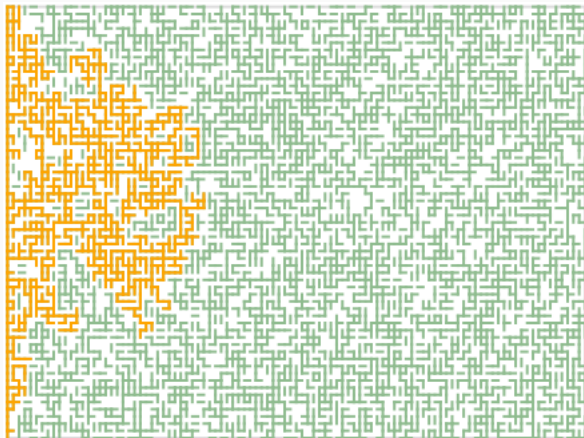
Du grain de café au grain de sable

$$p = 0,47$$



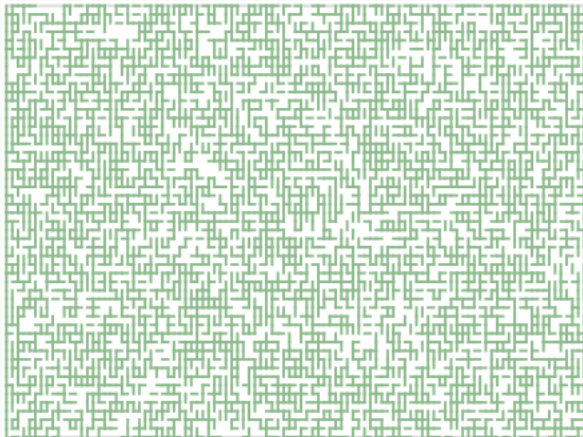
Du grain de café au grain de sable

34% de la roche parcourue



Du grain de café au grain de sable

$$p = 0,53$$



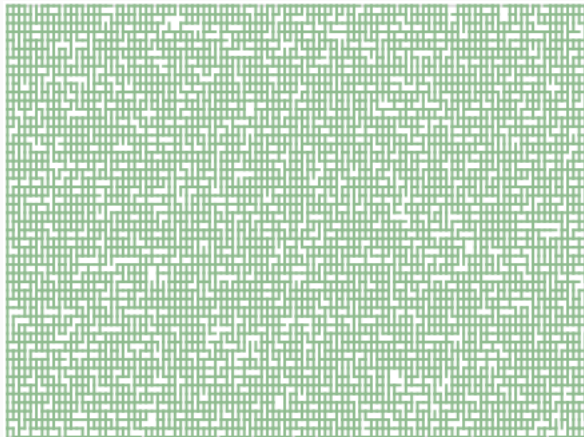
Du grain de café au grain de sable

100% de la roche parcourue



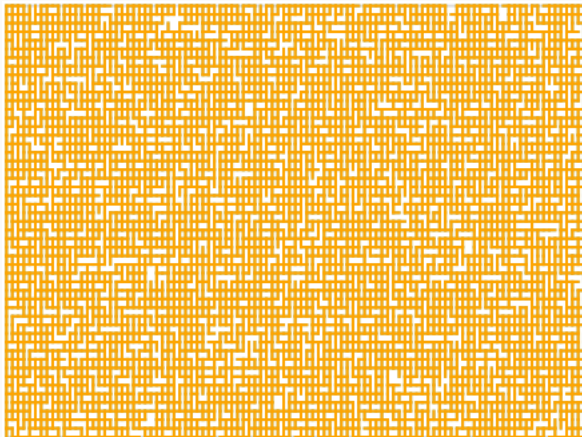
Du grain de café au grain de sable

$$p = 0,829$$



Du grain de café au grain de sable

100% de la roche parcourue



Un exemple de percolation

Histoire et définitions

Phénomène de seuil

Ouverture

La théorie de la percolation a été mise en place milieu 1950 par Simon BROADBENT et John HAMMERSLEY lors de recherches sur les masques à gaz.

Le *phénomène de percolation* est un effet de seuil : une faible variation de paramètres limite la transmission à quelques voisins proches ou bien arbitrairement lointains.

Le *phénomène de percolation* est un effet de seuil : une faible variation de paramètres limite la transmission à quelques voisins proches ou bien arbitrairement lointains.

On dit alors qu'il y a *percolation* si une infinité de nœuds fusionnent afin de créer un amas infini.

Le *phénomène de percolation* est un effet de seuil : une faible variation de paramètres limite la transmission à quelques voisins proches ou bien arbitrairement lointains.

On dit alors qu'il y a *percolation* si une infinité de nœuds fusionnent afin de créer un amas infini.

On distingue la *percolation de lien* qui attribue une probabilité de présence à un lien de la *percolation de site*.

Un exemple de percolation

Histoire et définitions

Phénomène de seuil

Ouverture

Un *modèle de percolation* par lien sur un graphe $G = (S, A)$ est une famille de variables aléatoires $(X_a)_{a \in A}$ à valeurs dans $\{0, 1\}$. Pour un lien $a \in A$, celui-ci est ouvert si $X_a = 1$ et fermé sinon. Une configuration sera notée $\omega = (\omega_a)_{a \in A} \in \{0, 1\}^A$.

Un *modèle de percolation* par lien sur un graphe $G = (S, A)$ est une famille de variables aléatoires $(X_a)_{a \in A}$ à valeurs dans $\{0, 1\}$. Pour un lien $a \in A$, celui-ci est ouvert si $X_a = 1$ et fermé sinon. Une configuration sera notée $\omega = (\omega_a)_{a \in A} \in \{0, 1\}^A$.

Une *percolation de Bernoulli de paramètre p* est une percolation où la famille de variables aléatoires est indépendante et identiquement distribuée et chaque variable suit une loi de Bernoulli de même paramètre $p \in [0; 1]$.

On travaillera avec l'espace probabilisé $(\Omega, \mathcal{A}, P_p)$ où

- $\Omega = \prod_{a \in A} \{0, 1\} = \{0, 1\}^A \simeq \mathcal{P}(A)$
- $\mathcal{A} = \mathcal{P}(\{0, 1\})^{\otimes A}$ est tribu produit
- $P_p = \prod_{a \in A} \mu_p$ où μ_p est la mesure sur $\{0, 1\}$ tel que $\mu_p(\{0\}) = 1 - p$ et $\mu_p(\{1\}) = p$

On muni Ω de la relation d'ordre suivante :

$\omega \preceq \omega'$ ssi toute arête ouverte de ω est ouverte dans ω' .

Ainsi $A \in \mathcal{A}$ est dit croissant si

$$\forall (\omega, \omega') \in A \times \Omega \quad \omega \preceq \omega' \Rightarrow \omega' \in A.$$

Notations

$\{x \rightsquigarrow y\} := \{\omega \in \{0, 1\}^A \mid x \overset{\omega}{\rightsquigarrow} y\}$ l'évènement : « il existe un chemin entre $x \in S$ et $y \in S$. »

$\{x \rightsquigarrow \infty\}$ l'évènement : « il y a percolation en $x \in S$ », c'est à dire qu'il existe un amas infini contenant le point $x \in S$.

On remarque que ces évènements sont croissants.

$\theta_x : p \longmapsto P_p(\{x \rightsquigarrow \infty\})$ la probabilité qu'il y ait percolation en $x \in S$ pour le paramètre p .

$\Pi : p \longmapsto P_p(\bigcup_{x \in S} \{x \rightsquigarrow \infty\})$ la probabilité qu'il y ait percolation.

Existence d'une probabilité critique

Il existe une probabilité critique $p_c \in [0; 1]$ telle que pour tout $x \in S$

$$\begin{cases} \theta_x(p) = 0 & \text{si } p \leq p_c \\ \theta_x(p) > 0 & \text{si } p > p_c \end{cases} .$$

Démonstration de l'existence d'une probabilité critique

On définit une telle probabilité p_c comme

$$p_c := \sup(\{p \in [0; 1] \mid \theta_x(p) = 0\}) \text{ pour } x \in S.$$

Celle-ci existe bien car l'ensemble de départ est majoré et non vide, comme $\theta_x(0) = 0$.

Démonstration de l'existence d'une probabilité critique

On définit une telle probabilité p_c comme

$$p_c := \sup(\{p \in [0; 1] \mid \theta_x(p) = 0\}) \text{ pour } x \in S.$$

Celle-ci existe bien car l'ensemble de départ est majoré et non vide, comme $\theta_x(0) = 0$.

Soit $p \in [0; p_c[$ (sous réserve d'existence) :

$p < p_c$ et p_c est la borne supérieure de l'ensemble des zéros de θ_x donc p est alors un zéro de θ_x : il n'y a pas percolation.

Démonstration de l'existence d'une probabilité critique

On définit une telle probabilité p_c comme

$$p_c := \sup(\{p \in [0; 1] \mid \theta_x(p) = 0\}) \text{ pour } x \in S.$$

Celle-ci existe bien car l'ensemble de départ est majoré et non vide, comme $\theta_x(0) = 0$.

Soit $p \in [0; p_c[$ (sous réserve d'existence) :

$p < p_c$ et p_c est la borne supérieure de l'ensemble des zéros de θ_x donc p est alors un zéro de θ_x : il n'y a pas percolation.

Soit $p \in]p_c; 1]$ (sous réserve d'existence) :

$p > p_c$ donc p n'est pas un zéro de θ_x , et comme il s'agit d'une probabilité, elle est strictement positive.

Démonstration de l'existence d'une probabilité critique

Inégalité FKG (Fortuin, Kasteleyn, Ginibre)

Soit J au plus dénombrable dont chaque élément est dans l'état 0 avec une probabilité p ou bien 1 avec une probabilité $1 - p$.

On pose $\Omega = \{0, 1\}^J$.

Soient deux parties croissantes (au sens de $\preceq$) A et B de Ω , alors

$$P(A \cap B) \geq P(A)P(B).$$

On admet ce résultat.

Démonstration de l'existence d'une probabilité critique

Soit $(x, y) \in S^2$.

$$\begin{aligned}\theta_x(p) &= P_p(\{x \longleftrightarrow \infty\}) \\ &\geq P_p(\{x \longleftrightarrow y\} \cap \{y \longleftrightarrow \infty\}) \\ &\geq P_p(\{x \longleftrightarrow y\}) \cdot P_p(\{y \longleftrightarrow \infty\}) \\ &\quad \text{car } \{x \longleftrightarrow y\} \text{ et } \{y \longleftrightarrow \infty\} \text{ sont croissants donc l'inégalité FKG s'applique} \\ &= \underbrace{P_p(\{x \longleftrightarrow y\})}_{>0} \cdot \theta_y(p)\end{aligned}$$

Ainsi θ_x et θ_y s'annulent mutuellement.



Une transition de phase

La probabilité de percolation sur un réseau infini vérifie

$$\Pi(p) = \begin{cases} 0 & \text{si } p \leq p_c \\ 1 & \text{si } p > p_c \end{cases}$$

d'où l'appellation "transition de phase".

Tribu terminale ou asymptotique

Soit $(\mathcal{A}_k)_{k \in \mathbb{N}}$ une suite de tribus indépendantes sur $(\Omega, \mathcal{A}, P)$.

On note $\mathcal{B}_n$ la tribu engendrée par $(\mathcal{A}_k)_{k \geq n}$.

La tribu $\mathcal{B}_\infty = \bigcap_{n \in \mathbb{N}} \mathcal{B}_n$ est appelée tribu terminale.

Tribu terminale ou asymptotique

Soit $(\mathcal{A}_k)_{k \in \mathbb{N}}$ une suite de tribus indépendantes sur $(\Omega, \mathcal{A}, P)$.

On note $\mathcal{B}_n$ la tribu engendrée par $(\mathcal{A}_k)_{k \geq n}$.

La tribu $\mathcal{B}_\infty = \bigcap_{n \in \mathbb{N}} \mathcal{B}_n$ est appelée tribu terminale.

Une autre définition de la tribu asymptotique est qu'elle est constituée des évènements dit de queue, i.e. dont la réalisation dépend d'une suite de variables aléatoires indépendantes mais ne dépend d'aucun sous-ensemble fini de ces variables.

Loi du zéro-un de Kolmogorov

Tout événement de la tribu terminale est de probabilité 0 ou 1.

Démonstration de la loi du zéro-un de Kolmogorov

Soit $B \in \mathcal{B}_\infty$. On considère la **classe monotone** des évènements indépendants de B :

$$\mathcal{M}_B = \{A \in \mathcal{A} \mid P(A \cap B) = P(A)P(B)\}.$$

Démonstration de la loi du zéro-un de Kolmogorov

Soit $B \in \mathcal{B}_\infty$. On considère la **classe monotone** des évènements indépendants de B :

$$\mathcal{M}_B = \{A \in \mathcal{A} \mid P(A \cap B) = P(A)P(B)\}.$$

Montrons que $\mathcal{B}_\infty \subset \mathcal{M}_B$, ainsi on aura B indépendant de lui-même, i.e. $P(B) = P(B \cap B) = P(B)^2$ et $P(B) \in \{0, 1\}$.

Démonstration de la loi du zéro-un de Kolmogorov

On pose $\mathcal{C}_n := \sigma(\mathcal{A}_0, \dots, \mathcal{A}_n)$.

Les tribus de départ étant indépendantes, on a pour tout $n \in \mathbb{N}$ que $\mathcal{C}_n$ et $\mathcal{B}_{n+1} = \sigma(\mathcal{A}_k \mid k \geq n+1)$ le sont aussi.

Démonstration de la loi du zéro-un de Kolmogorov

On pose $\mathcal{C}_n := \sigma(\mathcal{A}_0, \dots, \mathcal{A}_n)$.

Les tribus de départ étant indépendantes, on a pour tout $n \in \mathbb{N}$ que $\mathcal{C}_n$ et $\mathcal{B}_{n+1} = \sigma(\mathcal{A}_k \mid k \geq n+1)$ le sont aussi.

Par conséquent les tribus $\mathcal{C}_n$ et $\mathcal{B}_\infty$ sont indépendantes pour tout $n \in \mathbb{N}$:

$$\forall A \in \bigcup_{n=1}^{\infty} \mathcal{C}_n \quad \forall B \in \mathcal{B}_\infty \quad P(A \cap B) = P(A)P(B).$$

Démonstration de la loi du zéro-un de Kolmogorov

On pose $\mathcal{C}_n := \sigma(\mathcal{A}_0, \dots, \mathcal{A}_n)$.

Les tribus de départ étant indépendantes, on a pour tout $n \in \mathbb{N}$ que $\mathcal{C}_n$ et $\mathcal{B}_{n+1} = \sigma(\mathcal{A}_k \mid k \geq n+1)$ le sont aussi.

Par conséquent les tribus $\mathcal{C}_n$ et $\mathcal{B}_\infty$ sont indépendantes pour tout $n \in \mathbb{N}$:

$$\forall A \in \bigcup_{n=1}^{\infty} \mathcal{C}_n \quad \forall B \in \mathcal{B}_\infty \quad P(A \cap B) = P(A)P(B).$$

Ceci entraîne que $\mathcal{C}_\infty := \bigcup_{n=1}^{\infty} \mathcal{C}_n \subset \mathcal{M}_B$ et par **théorème des classes monotones** $\sigma(\mathcal{C}_\infty) = \mathcal{M}(\mathcal{C}_\infty) \subset \mathcal{M}_B$.

Démonstration de la loi du zéro-un de Kolmogorov

D'autre part,

$$\forall k \in \mathbb{N} \quad \mathcal{A}_k \subset \mathcal{C}_k \subset \mathcal{C}_\infty \subset \sigma(\mathcal{C}_\infty) \subset \mathcal{M}_B$$

donc

$$\forall n \in \mathbb{N} \quad \mathcal{B}_n = \sigma(\mathcal{A}_k \mid k \geq n) \subset \sigma(\mathcal{C}_\infty) \subset \mathcal{M}_B$$

i.e.

$$\mathcal{B}_\infty \subset \mathcal{M}_B.$$



Démonstration de la transition de phase

L'événement $\bigcup_{x \in S} \{x \rightsquigarrow \infty\}$ est un événement de queue et donc, d'après la loi du zéro-un de Kolmogorov, $\Pi(p) \in \{0, 1\}$.

Démonstration de la transition de phase

L'événement $\bigcup_{x \in S} \{x \leftrightarrow \infty\}$ est un événement de queue et donc, d'après la loi du zéro-un de Kolmogorov, $\Pi(p) \in \{0, 1\}$.

Pour $p \in [0; p_c]$:

$$\Pi(p) \leq \sum_{x \in S} \theta_x(p) = 0 \quad \text{d'après l'inégalité de Boole}$$

d'où $\Pi(p) = 0$.

Démonstration de la transition de phase

Pour $p \in]p_c; 1]$:

$$\{y \leftrightarrow \infty\} \subset \bigcup_{x \in S} \{x \leftrightarrow \infty\}.$$

donc par croissance de P_p ,

$$0 < P_p(\{y \leftrightarrow \infty\}) = \theta_y(p) \leq P_p\left(\bigcup_{x \in S} \{x \leftrightarrow \infty\}\right) = \Pi(p)$$

et donc $\Pi(p) = 1$.

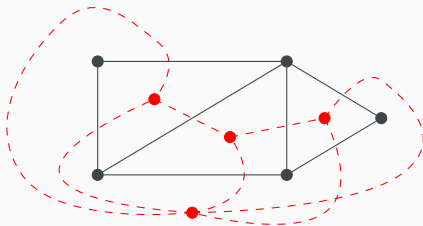


Théorème de Harris-Kesten (1960, 1980)

La probabilité critique pour une percolation de lien dans un réseau carré en dimension 2 vaut $p_c = \frac{1}{2}$.

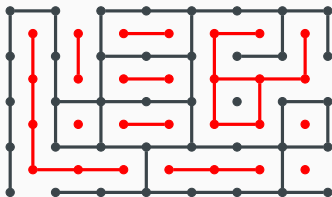
Remarque : pour une percolation de site dans un même réseau, $p_c \approx 0,5928$.

À un graphe non orienté planaire, on associe son **graphe dual**



Une idée de démonstration

Prenons pour convention que si une arête est ouverte dans le graphe, elle est fermée dans le dual et inversement.



Une idée de démonstration

Avec p_c^* la probabilité critique du graphe dual, il paraît intuitif que

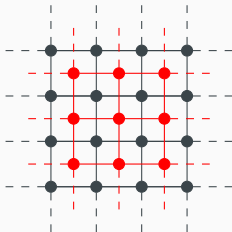
$$p_c^* = 1 - p_c.$$

Une idée de démonstration

Avec p_c^* la probabilité critique du graphe dual, il paraît intuitif que

$$p_c^* = 1 - p_c.$$

Ensuite, le graphe dual de $\mathbb{L}_2$ n'est autre que lui-même :



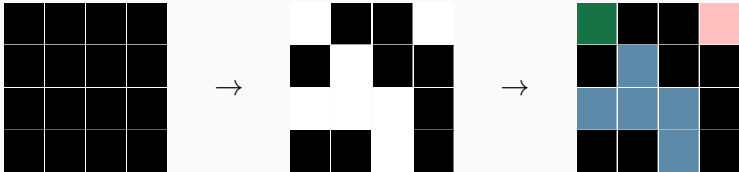
Ainsi $p_c = \frac{1}{2}$.



Théorème

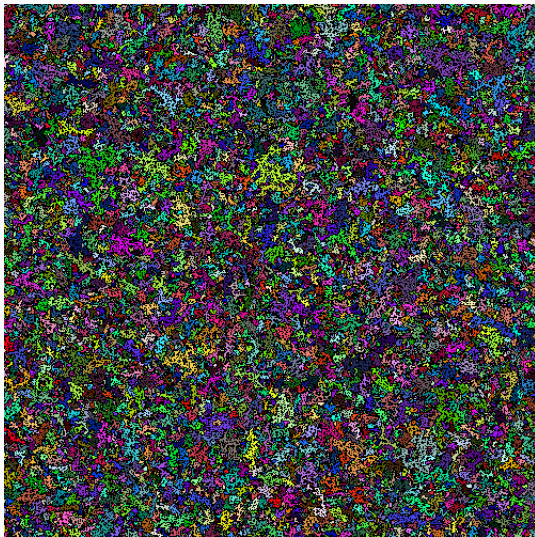
Pour $p > p_c$, il existe presque sûrement un unique amas infini sur $\mathbb{L}_2$.

Une percolation de site



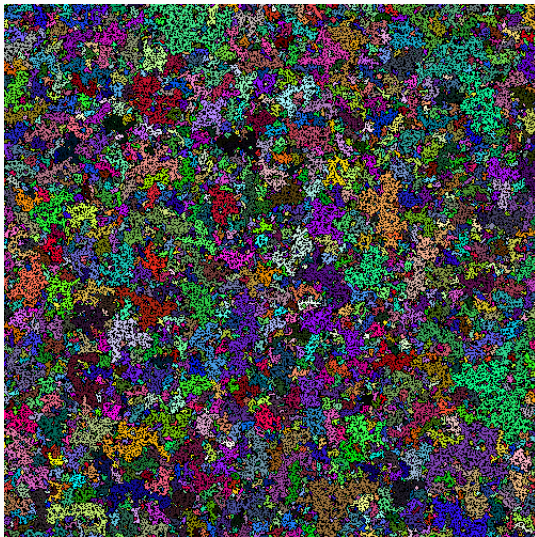
Une percolation de site

$$p = 0,50$$



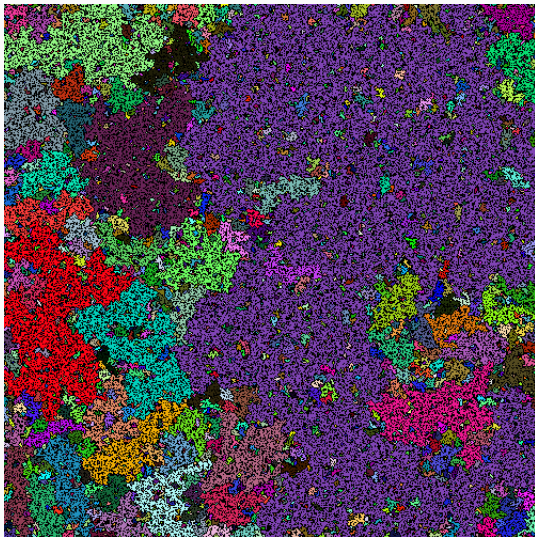
Une percolation de site

$$p = 0,55$$



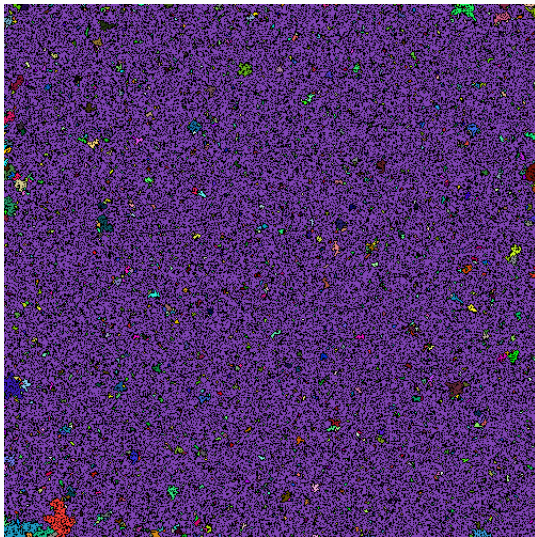
Une percolation de site

$$p = 0,60$$



Une percolation de site

$$p = 0,65$$



Un exemple de percolation

Histoire et définitions

Phénomène de seuil

Ouverture

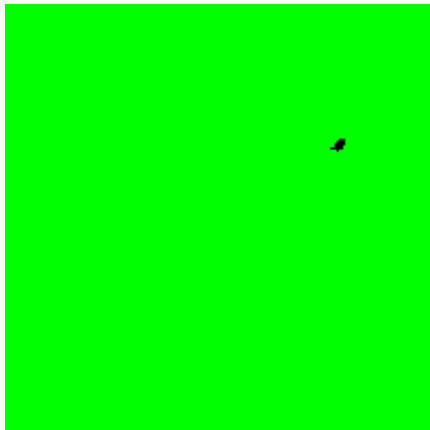
Applications

- la modélisation de feux de forêt ;
- la modélisation d'épidémies ;
- la modélisation de réseaux de communication ;
- l'aimantation (modèle d'ISING) ;
- la transition sol-gel ;
- le passage de l'archipel d'îles au continent (Pierre-Gilles DE GENNES, 1976) ;
- la conduite du changement.

Annexes

Modélisation de feux de forêts

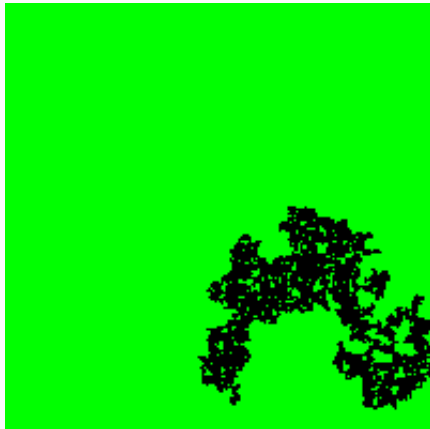
$$p = 0,29$$



0,05% de la forêt brûle

Modélisation de feux de forêts

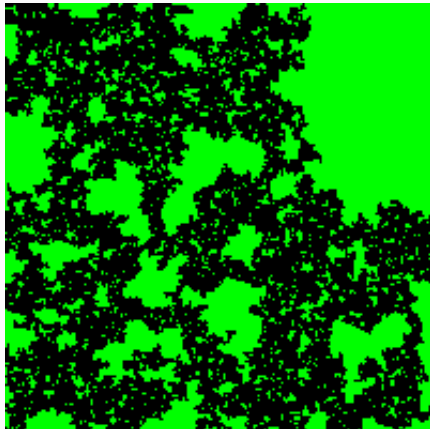
$$p = 0,49$$



10% de la forêt brûle

Modélisation de feux de forêts

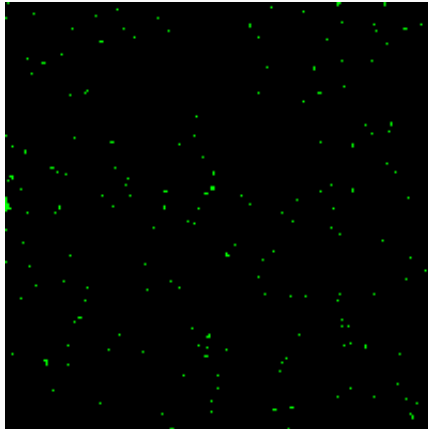
$$p = 0,51$$



55% de la forêt brûle

Modélisation de feux de forêts

$$p = 0,76$$



99,4% de la forêt brûle

Code de l'algorithme modélisant une roche poreuse i

```
# Analyses de percolations dans des réseaux carrés
# aléatoires modélisant l'écoulement d'eau dans une
# roche poreuse ou bien un percolateur de café.

import random
import matplotlib.pyplot as plt
import numpy as np
import os
os.system("clear")

# Modélisation de la recherche de la composante connexe
# dans un réseau carre aléatoire de taille n*m

def percolation(n, m, p, r=0) :

    # Modélisation du réseau
```


Code de l'algorithme modélisant une roche poreuse ii

```
grid = [list() for i in range(2*n+1)]
q = 0
for i in range(2*n+1) :
    grid[i] = [list() for j in range(2*m+1)]
    if i%2!=0 :
        grid[i][0].append([[0, 0], [n-i//2, n-i//2-1]])
        grid[i][0].append("darkseagreen")
    else :
        grid[i][0] = False
    for j in range(1, 2*m+1) :
        q = random.randint(1,100)/100
        grid[i][j] = list()
        if (i%2==1 and j%2==1) or (i%2!=1 and j%2!=1) ←
            :
            grid[i][j] = False
        else :
            # Vertical
```

Code de l'algorithme modélisant une roche poreuse iii

```
        if i%2==0 and j%2!=0 :
            grid[i][j].append([[j//2, j//2+1], [n-←
                i//2, n-i//2]])
# Horizontal
        elif i%2!=0 and j%2==0 :
            grid[i][j].append([[j//2, j//2], [n-i-←
                //2, n-i//2-1]])
        if q<p or p==1 :
            grid[i][j].append("darkseagreen")
        else :
            grid[i][j].append("white")

plt.plot([0, 0, m, m, m, m, 0], [n, n, n, n, 0, 0, 0], ←
        'gainsboro') # Cadre du réseau

for i in range(2*n+1) :
    for j in range(2*m+1) :
        if grid[i][j] != False :
```

Code de l'algorithme modélisant une roche poreuse iv

```
        if grid[i][j][1] != "white" :
            plt.plot(grid[i][j][0][0], grid[i][j↵
                ][0][1], color=grid[i][j][1])

# Calcul de la composante connexe
# Initialisation :
for i in range(2*n+1) :
    if i%2!=0 :
        if grid[i][0][1]=="darkseagreen" :
            grid[i][0][1] = 'Orange'

stack = list()
visited = [[False for j in range(2*m+1)] for i in ↵
    range(2*n+1)]
stack.append([1, 0])
while stack != [] :
    ind = stack.pop()
    i = ind[0]
```

Code de l'algorithme modélisant une roche poreuse v

```
j = ind[1]
if visited[i][j] == False :
    if grid[i][j][1] == "darkseagreen" :
        grid[i][j][1] = "Orange"
# Vertical
if i%2!=0 and j%2==0 :
    if i-1>=0 and j-1>=0 and grid[i-1][j-1][1]!='white' :
        stack.append([i-1, j-1])
    if i-2>=0 and grid[i-2][j][1]!='white' :
        stack.append([i-2, j])
    if i-1>=0 and j+1<2*m+1 and grid[i-1][j+1][1]!='white' :
        stack.append([i-1, j+1])
    if i+1<2*n+1 and j+1<2*m+1 and grid[i+1][j+1][1]!='white' :
        stack.append([i+1, j+1])
```

Code de l'algorithme modélisant une roche poreuse vi

```
        if i+2<2*n+1 and grid[i+2][j][1]!='white' ←
            :
            stack.append([i+2, j])
        if i+1<2*n+1 and j-1>=0 and grid[i+1][j←
            -1][1]!='white' :
            stack.append([i+1, j-1])
# Horizontal
elif i%2==0 and j%2!=0 :
    if j-2>=0 and grid[i][j-2][1]!='white' :
        stack.append([i, j-2])
    if i-1>=0 and j-1>=0 and grid[i-1][j←
        -1][1]!='white' :
        stack.append([i-1, j-1])
    if i-1>=0 and j+1<2*m+1 and grid[i-1][j←
        +1][1]!='white' :
        stack.append([i-1, j+1])
    if i+1<2*n+1 and j-1>=0 and grid[i+1][j←
        -1][1]!='white' :
```

Code de l'algorithme modélisant une roche poreuse vii

```
        stack.append([i+1, j-1])
    if i+1<2*n+1 and j+1<2*m+1 and grid[i+1][j←
        +1][1]!='white' :
        stack.append([i+1, j+1])
    if j+2<2*m+1 and grid[i][j+2][1]!='white' ←
        :
        stack.append([i, j+2])
visited[i][j] = True

# Donnée de l'atteinte du bord droit par la composante←
# connexe
goal = False
for i in range(2*n+1) :
    if grid[i][2*m] != False :
        if grid[i][2*m][1] == "Orange" :
            goal = True

# Calcul de la profondeur de la composante connexe
```

```
depht = 0
for j in range(2*m+1) :
    for i in range(2*n+1) :
        if grid[i][j] != False :
            if grid[i][j][1] == "Orange" :
                depht = j//2

# Affichages
print("Probabilité d'ouverture d'un lien p =", p)
print("Profondeur de la composante connexe ouverte :", ←
      depht, "\n\n")
title = "Probabilité d'ouverture d'un lien p = " + str ←
      (p)
plt.axis(False)
plt.title(title)
nom = 'percolation_eau_0'+str(r)+''.svg'
plt.savefig(nom)
```

Code de l'algorithme modélisant une roche poreuse ix

```
plt.pause(0.1)
for j in range(2*m+1) :
    for i in range(2*n+1) :
        if grid[i][j] != False :
            if grid[i][j][1] == "Orange" :
                plt.plot(grid[i][j][0][0], grid[i][j][0][1], 'Orange')

plt.draw()
plt.pause(1)

nom2 = 'percolation_eau_'+str(r)+'.svg'
plt.savefig(nom2)

plt.clf()
return goal
```


Code de l'algorithme modélisant une percolation de site i

```
# Algorithme modélisant une percolation de site

import random
import matplotlib.pyplot as plt
import numpy as np
from PIL import Image
from PIL import GifImagePlugin
import os
os.system("clear")

# Fonction qui crée un tableau de n couleurs
# distinctes ni noir, ni blanche

def colors_tab(n) :
```

Code de l'algorithme modélisant une percolation de site i

```
colors = list()
for i in range(n) :
    r = 0
    g = 0
    b = 0
    while (r,g,b) == (0,0,0) or (r,g,b) == ↵
        (255,255,255) or (r,g,b) == (203,51,255) or (r↵
        ,g,b) in colors :
        r = random.randint(0,255)
        g = random.randint(0,255)
        b = random.randint(0,255)
    colors.append((r,g,b))
return colors
```

```
# Modélisation d'une percolation de Bernoulli
# de paramètre p
```

Code de l'algorithme modélisant une percolation de site iii

```
def percolation(n, p, r='') :  
  
    print("\nProbabilité p =", p, "\n")  
  
    # Modélisation du réseau de la forêt  
    grid = [list() for i in range(n)]  
    for i in range(n) :  
        grid[i] = [list() for j in range(n)]  
        for j in range(n) :  
            q = random.randint(1,100)/100  
            if q < p or p==1 :  
                grid[i][j] = 0  
            else :  
                grid[i][j] = 1  
  
    # Système sans mise en évidence des différents amas
```

Code de l'algorithme modélisant une percolation de site iv

```
im = Image.new('RGB',(n,n))
for i in range(n):
    for j in range(n):
        if grid[i][j] == 0:
            im.putpixel((i,j),(203,51,255))
        else :
            im.putpixel((i,j),(0,0,0))

# Recherche des amas
clusters = list()
in_cluster = [[False for j in range(n)] for i in range(n)]
tmp = list()

stack = list()
visited = [[False for j in range(n)] for i in range(n)]
```


Code de l'algorithme modélisant une percolation de site vi

```
# Droite
if i+1<n and grid[i+1][j] == 0 and ←
    in_cluster[i+1][j] == False :
    stack.append([i+1, j])
    tmp.append([i+1, j])

# Bas
if j-1>=0 and grid[i][j-1] == 0 ←
    and in_cluster[i][j-1] == ←
    False :
    stack.append([i, j-1])
    tmp.append([i, j-1])

# Haut
if j+1<n and grid[i][j+1] == 0 and ←
    in_cluster[i][j+1] == False :
```

Code de l'algorithme modélisant une percolation de site **vii**

```
                stack.append([i, j+1])
                tmp.append([i, j+1])
                visited[i][j] = True

            clusters.append(tmp)
            tmp = list()

    colors = colors_tab(len(clusters))
    color = (128,68,181)

    maxi = 0
    ind_maxi = 0
    for l in range(len(clusters)) :
        if len(clusters[l]) > maxi :
            maxi = len(clusters[l])
            ind_maxi = l
```

Code de l'algorithme modélisant une percolation de site **viii**

```
for l in range(len(clusters)) :  
    while clusters[l] != [] :  
        ind = clusters[l].pop()  
        i = ind[0]  
        j = ind[1]  
        if l != ind_maxi :  
            im.putpixel((i,j), colors[l])  
        else :  
            im.putpixel((i,j), color)  
  
titre = 'image_'+str(r)+'_'+str(p)+'.png'  
im.save(titre, 'png')
```


Code de l'algorithme modélisant un feu de forêt i

```
# Analyses de percolations dans un réseau carré
# modélisant un feu de forêt.

import random
import matplotlib.pyplot as plt
import numpy as np
from PIL import Image
from PIL import GifImagePlugin
import os
os.system("clear")

# Modélisation de la propagation du feu.

def percolation(n, m, p, r='') :
```

Code de l'algorithme modélisant un feu de forêt ii

```
print("\nProbabilité de transmission du feu d'un arbre↵  
à l'autre :", p)  
  
# Modélisation du réseau de la forêt  
grid = [list() for i in range(n)]  
for i in range(n) :  
    grid[i] = [list() for j in range(m)]  
    for j in range(m) :  
        grid[i][j] = 0  
burning = list()  
neighbor = list()  
burnt = list()  
  
# Forêt sans feu  
im = Image.new('RGB', (n,m))  
for i in range(n):  
    for j in range(m):  
        im.putpixel((i,j), (0,255,0))
```

Code de l'algorithme modélisant un feu de forêt iii

```
im.save('foret_0.png', 'png')

# Premier arbre en feu
x = random.randint(0,n-1)
y = random.randint(0,m-1)
burning.append([x, y])
grid[x][y] = 1
im.putpixel((x,y),(194,24,7))
im.save('foret_1.png', 'png')

# Percolation
alpha = 2
while burning != [] or neighbor != [] :
    while burning != [] :
        ind = burning.pop(0)
        x = ind[0]; y = ind[1]
```

Code de l'algorithme modélisant un feu de forêt iv

Gauche

```
if x-1>=0 and grid[x-1][y] == 0:  
    q = random.randint(1, 100)/100  
    if q<p or p==1 :  
        neighbor.append([x-1, y])  
        grid[x-1][y] = 1  
        im.putpixel((x-1,y),(255,0,0))
```

Droite

```
if x+1<n and grid[x+1][y] == 0:  
    q = random.randint(1, 100)/100  
    if q<p or p==1 :  
        neighbor.append([x+1, y])  
        grid[x+1][y] = 1  
        im.putpixel((x+1,y),(255,0,0))
```

Bas

```
if y-1>=0 and grid[x][y-1] == 0:
```

Code de l'algorithme modélisant un feu de forêt v

```
q = random.randint(1, 100)/100
if q<p or p==1 :
    neighbor.append([x, y-1])
    grid[x][y-1] = 1
    im.putpixel((x,y-1),(255,0,0))

# Haut
if y+1<m and grid[x][y+1] == 0:
    q = random.randint(1, 100)/100
    if q<p or p==1 :
        neighbor.append([x, y+1])
        grid[x][y+1] = 1
        im.putpixel((x,y+1),(255,0,0))

burnt.append(ind)
grid[x][y] = 2
im.putpixel((x,y),(0,0,0))
```

Code de l'algorithme modélisant un feu de forêt vi

```
        burning = neighbor
        neighbor = list()
        titre = 'foret_'+str(alpha)+'.png'
        im.save(titre, 'png')
        alpha += 1

alive = 0
burnt2 = 0
for i in range(n) :
    for j in range(m) :
        if grid[i][j] == 0 :
            alive += 1
        elif grid[i][j] == 2 :
            burnt2 += 1

print("Au total,", burnt2, "arbre(s) brulé(s) et", ←
      alive, "arbre(s) encore vert.\n")
```

Code de l'algorithme modélisant un feu de forêt vii

```
# Création d'un GIF
```

```
imgs = list()
```

```
im1 = Image.open('foret_0.png')
```

```
for i in range(1, alpha) :
```

```
    noms = 'foret_' + str(i) + '.png'
```

```
    imgs.append(Image.open(noms))
```

```
im1.save('percolation_foret_' + str(r) + '.gif', save_all=↵  
True, append_images=imgs, duration=alpha/3, loop=↵  
=0, optimize=True, include_color_table=True, ↵  
interlace=True)
```

- [1] BERGLUND (N) : Probabilités et Physique Statistique, *28 juin 2013*.
- [2] SPECTRAL COLLECTIVE : Percolation: a Mathematical Phase Transition, *9 août 2022*.
- [3] JARRY (M), CAMPET (P) : Modélisation d'un feu de forêt, *Juin 2010*.
- [4] LESBRE (D) : Rapport_TIPE_Percolation, *2018*.
- [5] RUCH (J), CHABANOL (M-L) : RAPPELS de PROBABILITÉ.